Région Hauts-de-France
Cofinancé par l'Union européenne
realconseil DIAGNOSTIC ACCOMPAGNEMENT FORMATION

Documentation TWIG

1 Twig, c'est quoi exactement ?

Twig est un moteur de templates : tu écris des fichiers `.twig` (souvent du HTML + des balises Twig) et Twig les "rend" en HTML final à partir de données (variables) envoyées par ton code PHP (ou ton contrôleur Symfony).

Objectifs :

- séparer clairement **affichage** et **logique métier**
- éviter le "PHP spaghetti" dans les vues
- avoir un langage de template sûr (échappement XSS par défaut en HTML)

---

2 Les 3 syntaxes Twig à connaitre

Affichage (print)

Affiche une valeur (variable, expression, résultat d'un filtre, etc.) :

```
<h1>{{ title }}</h1>
```

Instructions (tags)

Pour la logique : `if`, `for`, `set`, `include`, `extends`, etc. :

```
{% if user %}Bonjour{% endif %}
```

Commentaires

```
{# Ceci est un commentaire Twig #}
```

---

3 Variables, accès aux données, valeurs par défaut

Variables simples

```
{{ name }}
{{ user }}
```

Accéder à des champs

Twig supporte plusieurs “formes” et choisit intelligemment :

- tableau : `user['email']`
- objet : `user.email` (propriété) ou `getEmail()` / `isAdmin()` si méthodes

Exemples :

```
{{ user.email }}
{{ user.profile.firstName }}
{{ product.price }}
```

Valeur par défaut

- Avec `default` :

```
{{ user.nickname|default('Invité') }}
```

- Avec l’opérateur `??` (coalesce) :

```
{{ user.nickname ?? 'Invité' }}
```

Définir une variable

```
{% set total = 0 %}
{% set title = 'Accueil' %}
```

---

4 Types, littéraux, opérateurs

Chaînes, nombres, booléens

```
{% set s = 'hello' %}
{% set n = 42 %}
{% set ok = true %}
{% set nothing = null %}
```

Tableaux (listes) et objets (hash)

```
{% set tags = ['php', 'twig', 'symfony'] %}
{% set user = { id: 10, name: 'Manu' } %}

{{ tags[0] }}            {# php #}
{{ user.name }}          {# Manu #}
```

Concaténation

```
{{ 'Bonjour ' ~ user.name }}
```

Opérateurs utiles

```
{% if a and b %}{% endif %}
{% if a or b %}{% endif %}
{% if not a %}{% endif %}

{% if n > 10 %}{% endif %}
{% if n in [1, 2, 3] %}{% endif %}
```

---

5 Conditions : if / elseif / else

```
{% if user %}
  Bonjour {{ user.name }}
{% elseif guest %}
  Bonjour invité
{% else %}
  Bonjour inconnu
{% endif %}
```

Comparaisons, “is”

Twig a un système de tests :

```
{% if user is null %}{% endif %}
{% if name is empty %}{% endif %}
{% if n is even %}{% endif %}
{% if n is odd %}{% endif %}
{% if email is defined %}{% endif %}
```

`defined` est très utile si tu n'es pas sûr qu'une variable existe.

---

6 Boucles : for, loop, else

Boucle simple

```
<ul>
  {% for p in products %}
    <li>{{ p.name }} - {{ p.price }} €</li>
  {% endfor %}
</ul>
```

Variables de boucle `loop`

```
{% for p in products %}
  {{ loop.index }} / {{ loop.length }} - {{ p.name }}
{% endfor %}
```

`loop` propose notamment :

- `loop.index` (1..n)
- `loop.index0` (0..n-1)
- `loop.first`, `loop.last`
- `loop.length`
- `loop.parent` (dans les boucles imbriquées)

For avec else (si liste vide)

```
{% for p in products %}
  <li>{{ p.name }}</li>
{% else %}
  <li>Aucun produit</li>
{% endfor %}
```

Itérer sur un hash

```
{% for key, value in config %}
  <p>{{ key }} = {{ value }}</p>
{% endfor %}
```

---

7 Filtres : la "boite à outils" Twig

Un filtre transforme une valeur : `valeur|filtre(...)`

Filtres courants

```
{{ name|upper }}
{{ name|lower }}
{{ name|capitalize }}
{{ text|trim }}
{{ text|length }}
{{ text|slice(0, 50) }}
```

HTML et sécurité : escape, raw

En HTML, Twig échappe par défaut :

```
{{ user.bio }}   {# échappé, donc safe contre XSS #}
```

Si tu veux afficher du HTML déjà sûr (attention) :

```
{{ post.content|raw }}
```

Dans un projet réel, `raw` doit être rare et justifié.

Date

```
{{ createdAt|date('Y-m-d H:i') }}
```

Number / format

```
{{ price|number_format(2, ',', ' ') }}
```

join

```
{{ tags|join(', ') }}
```

json_encode (pratique pour JS)

```

```

```
<script>
  const data = {{ payload|json_encode|raw }};
</script>
```

---

8 Fonctions Twig utiles

Twig a des fonctions (comme en PHP) :

```
{{ dump(user) }}              {# debug en dev (Symfony) #}
{{ include('parts/_card.html.twig') }}
{{ random(['a','b','c']) }}
{{ range(1, 10)|join(' ') }}
```

En Symfony, tu as en plus :

- `path('route_name', {id: 10})` pour générer une URL
- `asset('images/logo.png')` pour les assets
- `csrf_token('intention')` pour CSRF
- `form_*` pour les formulaires (si Forms)

Exemples Symfony :

```
<a href="{{ path('app_product_show', {id: product.id}) }}">Voir</a>
<img src="{{ asset('images/logo.svg') }}" alt="Logo">
```

---

9 Héritage de templates : extends + blocks

C'est le coeur de Twig. Tu fais un layout (base) puis des pages qui héritent.

base.html.twig

```
<!doctype html>
<html>
  <head>
    <meta charset="utf-8">
    <title>{% block title %}Mon site{% endblock %}</title>
  </head>
  <body>
    <header>{% block header %}Header{% endblock %}</header>

    <main>
      {% block body %}{% endblock %}
    </main>
```

```
    <footer>{% block footer %}Footer{% endblock %}</footer>
  </body>
</html>
```

page.html.twig

```
{% extends 'base.html.twig' %}

{% block title %}Accueil{% endblock %}

{% block body %}
  <h1>Bienvenue</h1>
  <p>{{ message }}</p>
{% endblock %}
```

Ajouter au contenu du parent : parent()

```
{% block header %}
  {{ parent() }}
  <nav>...</nav>
{% endblock %}
```

---

10 include, embed, blocks réutilisables

include simple

```
{{ include('parts/_flash.html.twig') }}
```

include avec variables

```
{{ include('parts/_card.html.twig', { title: p.name, price: p.price }) }}
```

only (isoler le scope)

```
{{ include('parts/_card.html.twig', { title: 'X' }, only=true) }}
```

embed (include + possibilité de redéfinir des blocks)

`embed` est pratique quand un composant a des blocks internes :

```
{% embed 'ui/_panel.html.twig' %}
  {% block content %}
    <p>Contenu spécifique</p>
  {% endblock %}
{% endembed %}
```

---

11 Macros : “fonctions” de template

Macros = réutilisation de rendu (boutons, badges, tableaux, etc.).

macros/ui.html.twig

```
{% macro badge(text) %}
  <span class="badge">{{ text }}</span>
{% endmacro %}
```

Utilisation

```
{% import 'macros/ui.html.twig' as ui %}

{{ ui.badge('Nouveau') }}
```

Tu peux aussi `from ... import ...` :

```
{% from 'macros/ui.html.twig' import badge %}
{{ badge('OK') }}
```

---

12 Portée (scope) des variables

- Les variables définies via `{% set %}` sont dans le scope du template.
- Dans un `include`, tu peux contrôler ce que tu envoies.
- Dans un `for`, la variable de loop (`p`) n’existe que dans la boucle.

Exemple :

```
{% set x = 1 %}
{% for i in [1,2,3] %}
  {% set x = x + i %}
{% endfor %}
{{ x }}
```

Selon les versions et configurations, les comportements de scope peuvent surprendre. En pratique, évite les “accumulateurs” dans les vues si tu peux.

---

13 Gestion des espaces

Twig peut générer des espaces et retours à la ligne. Parfois tu veux les contrôler.

trim sur les tags

```
{%- if ok -%}
  OK
{%- endif -%}
```

Le `-` enlève les espaces autour.

---

14 Auto-escape et contextes (HTML, JS, URL)

En HTML, l’échappement est la norme. Si tu injectes dans du JS ou une URL, évite les bricolages :

- pour JS, passe par `json_encode|raw`
- pour URL, utilise les fonctions `path` / `url` de Symfony

---

15 Bonnes pratiques Twig

- Mets la logique métier dans le PHP / contrôleur, pas dans Twig.
- Twig : conditions d’affichage, boucles, mise en forme, composants.
- Évite `raw` sauf si tu sais exactement ce que tu fais.
- Utilise `extends` + `blocks` comme base, `include` pour les composants.
- Nomme tes partiels : `parts/_card.html.twig`, `ui/_button.html.twig`.

---

16 Mini “cheat sheet” (récap)

```
{# afficher #}
{{ var }}

{# logique #}
{% if ... %}{% endif %}
{% for x in xs %}{% else %}{% endfor %}

{# héritage #}
{% extends 'base.html.twig' %}
{% block body %}{% endblock %}

{# inclure #}
```

```
{{ include('parts/_x.html.twig', {a: 1}) }}

{# filtres #}
{{ name|upper }}
{{ createdAt|date('Y-m-d') }}
{{ text|default('N/A') }}

{# tests #}
{% if x is defined %}{% endif %}
{% if x is empty %}{% endif %}
```